



ULTIMATE SNAKE GAME

AI-Assisted Development Report

Muhammad Waleed-ur-Rehman

AI Tools: DeepSeek · ChatGPT · Claude · Google AI Studio | Deployment: Vercel

Project Quick Facts

Game Title	Ultimate Snake Game (Premium Edition)
Task	AI Generated Snake Game
AI Tools Used	DeepSeek (architecture) · ChatGPT (features) · Claude (visual design)
Building & Development Tool	Google AI Studio
Tech Stack	HTML5 Canvas · CSS3 · Vanilla JavaScript
Deployment (Game Application Link)	Vercel https://killer-ultimate-snake-game.vercel.app/
Github link	https://github.com/Muhammad-Waleed-ur-Rehman/Ultimate-Killer-Snake-Game
Prompt Iterations	3 major iterations across 3 AI platforms
Final Status	Production-deployed · Fully playable · Portfolio-ready

1. Executive Summary

This report documents the complete AI-assisted development journey of the Ultimate Snake Game — a production-ready, feature-rich web game deployed at killer-ultimate-snake-game.vercel.app. The project was built entirely using AI language models as the primary development tool, demonstrating structured prompt engineering, iterative refinement, and multi-model workflows in modern software development.

Item	Detail
Project Title	Ultimate Snake Game (Premium Board Game)
Task Reference	AI Generated Snake Game
AI Tools Used	DeepSeek, ChatGPT (GPT-4), Claude (Anthropic)
Building & Development Tool	Google AI Studio
Tech Stack	HTML5 Canvas, CSS3, Vanilla JavaScript (no frameworks, no libraries)
Deployment (Game Application Link)	Vercel — https://killer-ultimate-snake-game.vercel.app/
Github link	https://github.com/Muhammad-Waleed-ur-Rehman/Ultime-Killer-Snake-Game
Iterations	3 major prompt iterations across 3 AI platforms
Final Status	Fully deployed, playable, production-grade game

Key Achievement

By combining the strengths of three different AI models — DeepSeek for architecture, ChatGPT for feature ideation, and Claude for visual polish — the final game exceeded a typical student project in design quality, feature depth, and technical sophistication.

2. Project Overview & AI Tool Pipeline

The task required building a Snake Game using any AI tool, with complete freedom over design, features, mechanics, and tech stack. Rather than using a single AI tool, this project deliberately employed three AI platforms in sequence — each assigned a specific role in the development pipeline.

AI Platform	Role in Project	Key Contribution
DeepSeek	Architecture Lead	Full technical specification: gameplay mechanics, food types, power-ups, PWA setup, game modes, UX/UI design, controls, and deliverables list
ChatGPT (GPT-4)	Feature Ideation	AI-driven features (Adaptive Difficulty, AI Coach, AI Opponent), achievement systems, theme systems, accessibility settings, statistics dashboard

AI Platform	Role in Project	Key Contribution
Claude (Anthropic)	Visual & Code Polish	Cyberpunk neon-noir aesthetic, specific hex codes, particle effects, animation timings, Web Audio API integration, SERPENT.EXE branding
Google AI Studio	Game Builder	Received all 3 prompts; generated actual HTML/CSS/JS code iteratively through back-and-forth refinement until the final deployable game was produced

The three AI platforms (DeepSeek, ChatGPT, Claude) were used exclusively for **prompt research and ideation** — generating detailed specifications and feature lists. The actual game development and code generation was carried out in **Google AI Studio**, where the refined prompts were fed iteratively to build, test, and improve the final game.

3. Prompt Iterations — Step-by-Step

Each AI interaction was a deliberate prompt engineering decision. The prompts were structured, role-assigned, and progressively refined across three iterations to extract increasingly specific and implementable output.

Iteration 1 — DeepSeek: Architecture & Full Specification

Original Prompt Sent to DeepSeek

"Assume you are an expert and professional game developer. Build a snake game. Assume it includes everything which makes it perfect for any new user to play. Incorporate features, buttons, install, advanced level UX/UI design features. Include more options which you think best. Give a detailed prompt."

What the Prompt Achieved

DeepSeek responded by generating a full product requirements specification — a game design document of professional quality. The output covered:

- Core gameplay: 20x20 grid, 150ms default speed, wall-collision and self-collision rules with configurable options
- Five food types: Standard Apple (+10pts), Golden Apple (+50pts), Speed Boost, Slow Down, Bonus Fruit (score double)
- Three power-ups: Shield (one-time collision protection), Score Multiplier x2 (10 seconds), Shrink (remove 3 tail segments)
- Complete control scheme: Arrow keys/WASD, touch swipe, on-screen D-pad, gamepad API, accessibility one-tap mode
- PWA specification: manifest.json, service worker, offline asset caching, install prompt (beforeinstallprompt event)
- Four game modes: Classic (endless), Timed (2 min), Adventure (20 levels with obstacles and lives system), Zen Mode
- Sound design: 8-bit style SFX, background music, haptic feedback on mobile
- Technical requirements: 60 FPS target, modular JS architecture, Lighthouse PWA score >= 90

Prompt Engineering Technique: Role Assignment

The prompt opened with "Assume you are an expert and professional game developer" — a deliberate role assignment technique that primed the model to respond from a domain-expert perspective. This single framing decision dramatically elevated the quality and depth of the output from generic suggestions to professional-grade specification.

What Worked

- Role assignment produced an exceptionally detailed, professional-grade specification document
- Open-ended phrasing — "include more options which you think best" — gave creative latitude; DeepSeek added Zen Mode and haptic feedback unprompted
- Output was immediately structured and usable as a development blueprint

What Did Not Work

- No visual style was specified — DeepSeek defaulted to generic 'flat design with subtle gradients'
- Multiple tech stack options were offered (Phaser 3 or vanilla JS) without a clear commitment
- No code was generated — the output was specification-only, requiring further passes for implementation

Iteration 2 — ChatGPT: AI Features & User-Friendly Settings

Original Prompt Sent to ChatGPT

"What AI and advanced user-friendly features can I add to my snake game? Include maximum settings which are user-friendly. Follow best game developer practices."

What the Prompt Achieved

ChatGPT expanded the feature set by introducing an AI-driven layer absent from the DeepSeek specification:

- AI Opponent Snake: Computer-controlled snake using A*, BFS, or Dijkstra pathfinding algorithms that collects food, avoids walls, and grows dynamically
 - AI Coach System: Post-game performance analysis reporting Reaction Score, Path Efficiency, and Risk Level with personalized improvement tips
 - Adaptive Difficulty AI: Dynamic speed and obstacle adjustments — reduces difficulty if player dies quickly, increases if player dominates
 - Smart Enemy AI: Pattern detection that recognizes player tendencies (e.g., clockwise looping) and pre-emptively blocks likely routes
 - AI Replay Analysis: Post-game heatmap of player movement, mistake detection, and suggested improvements analogous to chess engine analysis
 - Statistics Dashboard: High score, average score, total games played, total playtime, score trend graphs, weekly improvement charts
 - Achievement System: Beginner (score 10), Hungry (eat 100 food), Survivor (survive 10 min), Legend (score 1000)
 - Theme System: Cyberpunk, Space, Jungle, Ocean, Desert visual environments
-

- Snake Customization: Fire Snake, Ice Snake, Robot Snake, Dragon Snake skins with unlock-based progression
- Accessibility: Color-blind modes (Protanopia, Deuteranopia, Tritanopia), reduced motion, high-contrast UI

Prompt Engineering Technique: Constraint + Expansion

The prompt combined a specific focus area ("AI and advanced user-friendly features") with a scope-expanding modifier ("maximum settings") and a quality anchor ("follow best game developer practices"). This triple combination directed the model toward professional AAA standards while allowing broad creative output.

What Worked

- Novel AI feature layer added — Adaptive Difficulty, AI Coach, and AI Opponent were not present in Iteration 1 and significantly elevated the project's ambition
- Structured output with clear categories made feature prioritization straightforward
- Achievement system table was directly implementable in code without further refinement

What Did Not Work

- Several AI features (reinforcement learning, computer vision mode, voice commands) were technically correct but impractical for a single-session HTML/JS implementation
- No codebase context was provided — ChatGPT responded to an abstract specification rather than actual code
- Remained conceptual — no implementation code generated

Iteration 3 — Claude: Visual Design & Implementation-Ready Prompt

Original Prompt Sent to Claude

"Give me a well- refined and detailed prompt for an impressive snake game production ready application."

What the Prompt Achieved

Claude generated a comprehensive, implementation-ready prompt with a specific visual identity and complete technical instructions — the most code-ready output of the three iterations:

- Defined aesthetic: Cyberpunk neon-noir — dark void background (#050508), electric cyan snake (#00f5ff), pulsating magenta food (#ff2d78)
- Typography: 'Share Tech Mono' from Google Fonts for a terminal/hacker aesthetic across all UI text
- Visual effects: Gradient snake body (bright cyan head to darker teal tail), directional head glow, food scale pulse with radiant bloom
- Particle system: 8-12 small square sparks exploding outward from food position on each eat
- Death animation: Red screen flash (200ms overlay) followed by snake disintegration sequence (segments scatter/fade over 500ms)
- SERPENT.EXE branding: Splash screen with the title SERPENT.EXE and an [INITIALIZE] terminal-style button

- Game Over overlay styled as a terminal command prompt with [RESTART] button
- HUD design: Score (live), High Score (localStorage), level/speed tier, directional arrow indicator
- Web Audio API: Synth blip on food eat, descending tone on death, ambient drone loop (low volume, M key toggle)
- Technical: requestAnimationFrame with fixed delta-time step, no external libraries, modular JS — game state / rendering / input / particles / audio
- Input system: Arrow keys + WASD, reverse-movement prevention, single-frame input buffering to prevent rapid-input self-collision

Prompt Engineering Technique: Minimal Seed Prompting

The prompt to Claude was intentionally short — Give me a well- refined and detailed prompt for an impressive snake game production ready application — designed to give Claude full creative latitude. This technique works when the model is trusted to apply deep domain expertise without constraint. The result was a complete, layered specification with exact hex codes, font names, animation timing values, and API suggestions directly usable in code generation.

What Worked

- The cyberpunk aesthetic gave the game a distinctive, professional visual identity far beyond generic student projects
- Specific hex codes (#050508, #00f5ff, #ff2d78) eliminated all ambiguity for implementation
- Animation timings (200ms flash, 500ms disintegration) were immediately programmable without estimation
- The output itself became a reusable prompt artifact — deployable across any capable AI coding platform

What Did Not Work

- Minimal seed prompting requires a highly capable model — the same approach would fail with less sophisticated AI
- Some effects (WebGL bloom, true GLSL shaders) required canvas-based approximations as fallbacks
- Audio required browser permission handling (autoplay policy) that added unexpected implementation complexity

Iteration 4 — Google AI Studio: Actual Game Development

Prompt Strategy Used:

All 3 prompts from DeepSeek, ChatGPT, and Claude were pasted into Google AI Studio sequentially. The workflow was:

1. Pasted DeepSeek spec → AI Studio generates initial game skeleton
 2. Pasted ChatGPT feature list → refine and add AI/settings features
 3. Pasted Claude cyberpunk prompt → apply visual identity and polish
 4. Back-and-forth iterations to fix bugs, improve mechanics, and finalize
-

4. Prompt Iteration Summary Table

The following table consolidates all three iterations for direct comparison:

Iteration & AI		Prompt Strategy	Outcome
Iteration 1	DeepSeek	Role assignment: 'Assume expert game developer' + open-ended scope ('include best options')	Full product spec: 5 food types, 3 power-ups, 4 game modes, PWA setup, control scheme — but generic visual style, no code
Iteration 2	ChatGPT	Constraint + Expansion: 'AI features + maximum user-friendly settings + best practices'	Novel AI layer: Adaptive Difficulty, AI Coach, AI Opponent, Achievements, Themes — but over-scoped (RL/CV impractical), conceptual only
Iteration 3	Claude	Minimal Seed: 'Give me a well- refined and detailed prompt for an impressive snake game production ready application'	Complete implementation blueprint: hex codes, font names, animation timings, SERPENT.EXE brand, Web Audio API — fully code-ready

5. What Worked Well

5.1 Role Assignment Technique (DeepSeek)

Opening the DeepSeek prompt with 'Assume you are an expert and professional game developer' was the single most impactful prompt engineering decision in the project. It shifted the model from generic assistant mode into domain-expert mode, producing output with professional depth: timing values, collision rule options, power-up mechanics, and PWA specifications that would typically originate from a senior game developer.

5.2 Three-AI Pipeline

Using three AI tools in sequence — each for a different development layer — was far more effective than relying on a single model. DeepSeek excelled at structured specification. ChatGPT excelled at feature breadth and industry best-practice references. Claude excelled at aesthetic precision and implementation-ready detail. The division of responsibility extracted the best capability from each model.

5.3 Progressive Specificity Across Iterations

The prompts became progressively more specific: from 'build a perfect game' (broad vision) to 'what AI features should I add' (feature scoping) to 'give me an impressive implementation prompt' (technical specification). This mirrors the structure of professional product development cycles — and produced commensurately professional results at each stage.

5.4 Creative Latitude via Open-Ended Phrasing

Phrases like 'include more options which you think best' (DeepSeek) and the minimal seed approach (Claude) gave the models explicit permission to contribute creatively rather than mechanically follow

instructions. Both cases produced features not directly requested: Zen Mode from DeepSeek and SERPENT.EXE branding from Claude — both of which significantly improved the final product.

5.5 Google AI Studio as a Code Generator:

Google AI Studio's code generation capability translated complex multi-platform prompt specifications directly into runnable HTML/CSS/JS. The ability to iterate back-and-forth within a single session — refining, fixing, and extending the game — compressed what would be days of manual coding into a single focused development session.

5.6 Vercel Deployment

Deploying to Vercel from a pure HTML/CSS/JS project required zero configuration. The no-framework approach combined with Vercel's static hosting capability meant the game could be live within minutes of code completion — a practical demonstration of deployment best practices for rapid-iteration development. Refer Game link: <https://killer-ultimate-snake-game.vercel.app/>

6. What Did Not Work

6.1 Absent Visual Specification in Iteration 1

The DeepSeek prompt contained no visual style direction, resulting in a generic 'flat design with subtle gradients' recommendation. For a game project where visual identity is critical to perceived quality, this was a significant gap — only resolved when Claude introduced the cyberpunk neon-noir identity in Iteration 3. Visual specification should be included from the very first prompt.

6.2 Over-Ambitious Features from ChatGPT

Without a feasibility constraint in the prompt, ChatGPT included reinforcement learning integration, computer vision mode, and voice command systems — all legitimate game features but well beyond a single-session HTML/JS implementation. Future prompts to ChatGPT should include an explicit scope boundary: 'features implementable in vanilla HTML/CSS/JS without external ML libraries.'

6.3 Missing Codebase Context

None of the three prompts referenced an existing codebase or the current implementation stage. This meant all three AIs responded to abstract specifications rather than actual code. Providing the current code as context — even a partial snippet — would have enabled AI models to produce implementation-level guidance rather than specification-level suggestions.

6.4 Browser Autoplay Policy for Web Audio

The audio features specified by Claude required the browser to have received a user gesture before any audio could play. Modern browsers block audio autoplay without prior user interaction. This required adding a click-to-start gate that was not anticipated in the original prompt, adding unexpected complexity to the game launch flow.

6.5 Canvas Glow Performance on Mobile

The bloom glow effects specified for food items and the snake head looked excellent on desktop but degraded noticeably on mid-range Android devices due to limited GPU acceleration for canvas shadow-blur operations. A performance-conditional implementation was required — full quality on desktop, simplified on mobile — which was not specified in the original prompts.

6.6 First-Pass Code from AI Studio Was Incomplete:

The initial code generated by Google AI Studio from the DeepSeek specification was functional but incomplete — several features (particle effects, audio, mobile controls) required multiple follow-up prompts to implement correctly. This confirms that even with a detailed spec, AI code generation requires iterative human guidance rather than single-shot execution.

7. How Prompts Improved Over Time

The three iterations demonstrate a clear, measurable evolution in prompt engineering sophistication across six dimensions:

Dimension	Iteration 1 (Early)	Iteration 3 (Final)
Specificity	'Build a perfect snake game' — broad, outcome-focused	Hex codes (#050508, #00f5ff), font names, ms-level animation timings — fully precise
Role Framing	No explicit persona — AI defaults to generic assistant	'Assume you are an expert game developer' — domain expert mode activated
Scope Control	No limits — AI included impractical RL and CV features	Implicit constraint: 'no external libraries, pure HTML/CSS/JS'
Visual Detail	No aesthetic specification — generic defaults produced	Full system: color palette, typography choice, particle counts, glow specs
Tech Stack	Multiple options (Phaser 3 or vanilla JS) — undecided	Single committed approach: vanilla JS for deployment simplicity
Output Format	Specification documents — planning level only	Implementation-ready prompts — directly usable in code generation

Core Lesson

The most powerful insight from this project: specificity removes ambiguity, but creative latitude produces unexpected value. The best prompts balanced both — giving the AI a defined role and constraints while allowing genuine creative contribution.

8. Challenges Faced & Solutions Applied

Challenge	Impact	Solution Applied
Conflicting specifications across three AI tools — each generated a slightly different feature set	High — manual reconciliation required to produce a coherent final spec	Created a priority hierarchy: DeepSeek spec as base architecture, ChatGPT features as additions, Claude aesthetic as the final overlay
Browser autoplay policy blocking Web Audio API on game launch	Medium — game launched silently, breaking the intended audio-first experience	Added a user-interaction gate (click-to-start screen) with a silent fallback mode and M key mute toggle
Canvas glow and shadow-blur degrading on mobile GPU	Medium — particle and glow effects visually degraded on mid-range Android	Implemented device detection: full shadow-blur quality on desktop, simplified flat rendering on mobile
Overly ambitious AI features from ChatGPT (RL, CV, voice commands)	Low — no time to implement within a vanilla JS single-session project	Applied must-have vs. nice-to-have filter; implemented Adaptive Difficulty only (highest impact, lowest complexity)
Generic visual design from early iterations	High — game resembled a template project rather than a portfolio piece	Full Iteration 3 with Claude dedicated to aesthetic specification — SERPENT.EXE cyberpunk identity resolved the problem entirely
PWA service worker and asset caching complexity	Medium — offline functionality required precise cache management	Used the DeepSeek specification as an implementation checklist; implemented manifest.json and cache-first service worker strategy
Integrating 3 separate AI prompt outputs into one coherent Google AI Studio build	High — conflicting design decisions required manual prioritization before feeding into AI Studio	Pasted prompts in order of priority: architecture first (DeepSeek), features second (ChatGPT), aesthetics last (Claude) — giving AI Studio a layered build sequence

9. Prompt Engineering Techniques Applied

This project consciously applied multiple established prompt engineering techniques. Each is documented below with its definition and direct application in the project:

Technique	Definition	Application in This Project
Role Assignment	Assigning the AI a specific expert persona before the task is issued	'Assume you are an expert and professional game developer' — Iteration 1, DeepSeek
Constraint + Expansion	Combining a specific focus area with a scope-expander and a quality anchor	'AI features + maximum user-friendly settings + best developer practices' — Iteration 2, ChatGPT

Technique	Definition	Application in This Project
Minimal Seed Prompting	Short, open-ended prompt to allow maximum AI creative contribution	'Give me a detailed prompt for an impressive snake game' — Iteration 3, Claude
Positive Anchoring	Asking the AI to apply known best practices as a quality reference	'Follow best game developer practices' anchored ChatGPT's output to AAA-standard expectations
Iterative Refinement	Using the output of one prompt as the conceptual context for the next	Claude's visual spec was layered on top of DeepSeek's feature list and ChatGPT's AI features
Output Type Specification	Explicitly requesting the format of the output, not just its content	'Give a detailed prompt' directed the AI to produce a reusable prompt document rather than raw code

10. Final Deployed Game — Features Overview

The game at killer-ultimate-snake-game.vercel.app integrates features from all three AI iterations into a single cohesive, production-deployed product:

Core Gameplay — from DeepSeek Iteration 1

- Grid-based snake movement with progressive speed increase every 5 food items
- Five food types: Standard Apple (+10pts), Golden Apple (+50pts, sparkle), Speed Boost, Slow Down, Bonus Fruit (2x score)
- Three power-ups: Shield (one collision skip), Score Multiplier x2 (10 seconds), Shrink (remove 3 tail segments)
- Four game modes: Classic (endless high-score), Timed (120 seconds), Adventure (20 levels with obstacles and 3 lives), Zen Mode (no death)
- Wall-wrap toggle, configurable self-collision, obstacle system in Advanced difficulty

AI & Advanced Features — from ChatGPT Iteration 2

- Adaptive Difficulty: game speed and obstacle density auto-adjust to player performance level
- Statistics Dashboard: high score per mode, games played, total food eaten, total playtime
- Achievement System: unlockable badges with defined point/action thresholds
- Multiple visual themes with player-selectable modes

Visual Design & Code Quality — from Claude Iteration 3

- Cyberpunk neon-noir identity: #050508 background, #00f5ff snake, #ff2d78 food, Share Tech Mono font
- Particle burst on food collection: 8-12 square sparks exploding from food position
- Death animation: red screen flash (200ms) + snake disintegration sequence (500ms scatter/fade)
- SERPENT.EXE splash screen and [INITIALIZE] / [RESTART] terminal-styled buttons
- Web Audio API: synth blip on eat, descending death tone, toggleable ambient drone loop

- PWA: manifest.json, service worker with offline asset caching, native install prompt
- Pure HTML/CSS/JS — no external frameworks, no libraries, no build tools required

Built & Deployed via Google AI Studio

- Game code generated entirely through Google AI Studio's AI code generation.
- Iterative back-and-forth development: multiple refinement rounds within a single AI Studio session.
- Final output: single-file HTML/CSS/JS deployed directly to Vercel.
- Live at: <https://killer-ultimate-snake-game.vercel.app/>

11. Key Learnings

11.1 AI as a Development Multiplier

The most significant learning from this project is that AI tools amplify developer judgment rather than replace it. A vague prompt produces a generic result. A precise, role-assigned, constrained prompt produces professional-grade output. The developer's skill in the AI era is now partly prompt architecture — knowing how to frame requests, assign roles, and scope output.

11.2 Multi-Model Workflows Are Superior to Single-Tool Approaches

No single AI tool excelled at everything. DeepSeek was strong at structured specification. ChatGPT was strong at feature breadth and industry reference. Claude was strong at aesthetic precision and implementation-ready detail. The three-model pipeline produced a result superior to any single-model approach would have achieved alone.

11.3 Iteration Is the Core Discipline

The first prompt never produces the final result. The project improved dramatically from Iteration 1 (generic specification, no visual identity) through Iteration 3 (branded, precisely specified, implementation-ready design). Prompt engineering is fundamentally an iterative craft — each output becomes context and motivation for a more refined next prompt.

11.4 Visual Identity Transforms Perceived Quality

The shift from a generic flat design to the SERPENT.EXE cyberpunk identity transformed the project from a functional student assignment to a portfolio-ready piece. A single dedicated iteration on visual specification delivered outsized returns in perceived quality, engagement, and differentiation from comparable student projects.

11.5 Scope Constraints Must Be Explicit

AI models default to comprehensive rather than achievable recommendations. Without explicit scope boundaries, ChatGPT included reinforcement learning and computer vision features that were impractical for the project timeline. Future prompts should always include a feasibility statement: 'features implementable in [tech constraint] within [time constraint].'

12. Project Summary

This task successfully demonstrates AI-assisted game development using a deliberate three-model pipeline: DeepSeek for architecture and specification, ChatGPT for feature depth and AI integration, and Claude for visual identity and implementation precision. The result is a production-deployed Snake game that exceeds baseline student project requirements in design quality, feature sophistication, and technical execution — achieved entirely through structured, iterative prompt engineering.